

Contents

Before the call (5 minutes)

Login credentials (dev/demo only)

The one-sentence pitch

Suggested flow (15–20 minutes)

Wow moments to make sure you hit

If asked "can you prove nobody edited the ledger after the fact"

Things to be upfront about (don't oversell)

Resetting between rehearsals

Client Demo Script — Receiving Workflow Walkthrough

Phase 1 of the Inventory Control System. This script walks a live demo of the one workflow that's fully built end to end: **Receiving** — goods arriving at the warehouse, counted, inspected, approved, and posted to a tamper-evident stock ledger.

Before the call (5 minutes)

- 1. Start the app: `npm run dev` (from the project folder).
- 2. Reset demo data so the Receiving list is empty and document numbers start

clean: `npm run db:reset-demo`.

- 1. Open `http://localhost:3000` and confirm the login page loads.
- 2. **Set up fast role-switching.** You'll play 6 different roles during the

walkthrough (guard, receiver, checker, supervisor, branch manager, encoder). The clean way to do this live: open a separate browser profile or an incognito/private window per role, log each one in ahead of time, and switch between windows/tabs as the story moves. Logging out and back in between every step works too, just slower.

- 1. Have this doc open on a second screen or printed — it's written so you can read it while sharing only the browser.

Heads-up on the idle lock: if you pause on one screen for more than a few minutes mid-demo, the system will ask for a PIN before the next posting action. That's not a bug — call it out as a feature (see "Wow moments" below) rather than getting caught off guard by it.

Login credentials (dev/demo only)

Role	Username	Password	PIN (for posting/approval steps)
Security Guard	<code>security_guard</code>	<code>Password123!</code>	<code>1234</code>
Warehouse Receiver	<code>warehouse_receiver</code>	<code>Password123!</code>	<code>1234</code>
Warehouse Checker	<code>warehouse_checker</code>	<code>Password123!</code>	<code>1234</code>
Warehouse Supervisor	<code>warehouse_supervisor</code>	<code>Password123!</code>	<code>1234</code>
Branch Manager	<code>branch_manager</code>	<code>Password123!</code>	<code>1234</code>
Encoder	<code>encoder</code>	<code>Password123!</code>	<code>1234</code>
Owner	<code>owner</code>	<code>Password123!</code>	<code>1234</code>

Same password for every role, dev-only. Production accounts will be named, real people with their own credentials — make that distinction explicit if the client asks.

The one-sentence pitch

"Every stock movement in this system is a signed, sequential, tamper-evident ledger entry — never a number someone quietly edited. If a figure is ever wrong, we can prove exactly when it changed, who changed it, and what it was before."

That's the core design principle behind everything you're about to show: **inventory must never change without a traceable business transaction.**

Suggested flow (15–20 minutes)

1. Gate Log — `warehouse` / role: Security Guard

Log in as `security_guard`, go to **Gate Log** → **Log entry**.

- Direction: `IN`
- Vehicle plate / driver name: anything realistic
- Click **Log entry**

Say: "Nothing gets counted until it's logged at the gate. This becomes the record we cross-check later if a delivery ever needs to be returned."

2. Draft the Receiving Report — role: Warehouse Receiver

Switch to `warehouse_receiver`. Go to **Receiving** → **New Receiving Report**.

- Pick the supplier (**Iloilo Plastics Wholesale Corp.**)
- DR number: anything, e.g. `DR-DEMO-001`
- PO reference: leave blank or fill it in — either is worth showing (see

the no-PO callback control below)

- Add a line: pick a SKU from the catalog (e.g. ****Trash Bag XL 25x35"** —

TRASHBAG-XL-PACK50**), expected qty, unit cost

- Submit

Say: "This is the paperwork stage — what we expect to receive, before anyone counts anything."

3. Receiver's count — role: Warehouse Receiver

On the report's detail page, **Step 4 — Submit your count (Receiver)** card is visible only to the Receiver. Enter a quantity, submit.

4. Checker's count — role: Warehouse Checker (blind)

Switch to `warehouse_checker`, open the same report. **Step 5 — Submit your count (Checker, blind)** — the page literally does not show the Receiver's number anywhere.

This is the wow moment for this step: "The Checker's screen has no way to see what the Receiver counted — not hidden with CSS, the API response for this role never includes it. Two people have to independently arrive at the same number for this to move forward without a supervisor stepping in."

- Enter the **same quantity** to show the smooth path, or a ****different**

quantity** to show the tie-break escalation (Step 5 becomes a tie-break card for the Supervisor, with a required witness). Pick whichever story fits the time you have — showing the mismatch path once is a strong moment if the client has ever had a shrinkage dispute.

5. Quality inspection — role: Warehouse Supervisor or Receiver

Step 6 — Quality inspection. Click **Pass**, or (Supervisor only) **Reject (quarantine)** to show the rejection path exists and diverts goods out of the normal flow.

6. Verify — role: Warehouse Supervisor

Steps 7–8 — Prepare & verify. Click **Verify receiving report**.

Optional pointed moment: if you're logged in as the same person who received the delivery, the button is replaced with a red message — "You received this delivery and cannot verify it yourself." Worth triggering once deliberately to show separation of duties is enforced by the system, not a policy on paper.

7. No-PO callback (only if you left PO reference blank in step 2)

If there's no PO on file, a **G-04 — Confirm supplier call-back** card blocks approval until someone clicks **Confirm call-back completed**.

Say: "If there's no purchase order backing a delivery, the system won't let it get approved until someone has actually called the supplier back on the number we have on file — not a number written on the delivery paperwork itself, which is exactly the kind of detail that gets forged."

8. Approve — role: Branch Manager

Switch to `branch_manager`. **Step 9 — Approve** card shows a PIN field. Enter `1234`, wait for the token, click **Approve**.

Say: "Being logged in isn't enough to approve money-moving actions. Every single approval needs a fresh PIN entered right before it — even if you approved something five minutes ago." Which role has to approve is also computed automatically from the peso value of the delivery against configurable thresholds — small deliveries need a Branch Manager, larger ones escalate to the Owner.

9. Encode & post — role: Encoder

Switch to `encoder`. **Step 10 — Encode & post.**

- Click to open the camera and capture a live photo (this uses the device's actual camera stream, not a file picker — there's no "choose from gallery" option anywhere on this screen).

- Enter PIN `1234`, wait for the token.
- Click **Post to ledger**.

Say: "This is the point of no return — the moment this delivery actually becomes stock on the books. It needs a live photo, a fresh PIN, and a sequential government-compliant document number, all captured in the same atomic step. If any one of those is missing, nothing posts — there's no partial state where inventory changed but the paperwork didn't, or vice versa."

The report now shows **Posted**, and the top of the page shows the assigned document number (e.g. `RR-IL0-2026-000001`).

Wow moments to make sure you hit

- Blind counting** (step 4) — the checker's screen provably cannot see the receiver's figures.

- **Can't verify your own work** (step 6) — trigger this once if you have time.
- **PIN re-auth on every single posting** — mention that this is separate from, and in addition to, normal login. Being logged in and *active* still isn't enough.
- **Approval routing by value** — different peso thresholds route to different approvers automatically; nothing hardcoded to one person.
- **Live camera, no gallery upload** — closes the easiest way someone could submit a stock photo instead of a real one.
- **Sequential document numbers** — every posted report gets the next number in a registered range; skipping or reusing a number is rejected, not just discouraged.

If asked "can you prove nobody edited the ledger after the fact"

Yes — every ledger row is cryptographically chained to the one before it (each row's hash includes the previous row's hash), so editing any historical row breaks the chain from that point forward and is mathematically detectable. ****Be upfront that this isn't wired to a click-through screen yet**** — it's a verified, tested capability at the database layer (there's an automated check that proves a tampered row is caught), but the one-click "verify ledger integrity" report for an Auditor role is a small follow-up, not something to demo live today. Don't improvise a live click that doesn't exist.

Things to be upfront about (don't oversell)

- This is running on a local machine right now, not a hosted URL — a real deployment (hosted database, real domain, real accounts) is a separate, deliberate next step once the client wants one.
 - Only **Receiving** is built end to end. Orders, POS, sales, and AR are later phases in the build plan — if asked, be clear this is the foundation, not the whole system yet.
 - Live capture closes the "pick an existing photo from your gallery" gap, but a genuinely determined person could still photograph a photo on another screen. That's a known, accepted limitation — the system compensates for it with a weekly sample-audit report comparing typed quantities against photos (not yet built, planned).
 - Approval peso thresholds shown today are placeholder defaults, not the client's actual policy — flag this if they ask about the specific numbers.
-

Resetting between rehearsals

Every full run posts a real report and burns a document number. Before re-running the same script (a rehearsal, or a second live demo), reset:

```
npm run db:reset-demo
```

This clears all receiving reports, ledger rows, and document numbers, but keeps the catalog, suppliers, and logins intact — the app comes back to exactly the state described in this script.